# RideLink – Technical Report

IT3130 Application Development – Group Assignment

RideLink is a ride-hailing platform built as four independent microservices.

Repository: https://github.com/PasinduUmega/RideLink---AD-project---IT3130

| Service | Primary owner | Student ID |
|---|---|---|
| account-service | Wimalasooriya R.M.T.K | IT24102880 |
| driver-vehicle-service | Abeykoon K.A.A.R | IT24102787 |
| ride-management-service | Madushan E.P | IT24102952 |
| fare-payment-service | Fernando G.P.U | IT24102906 |

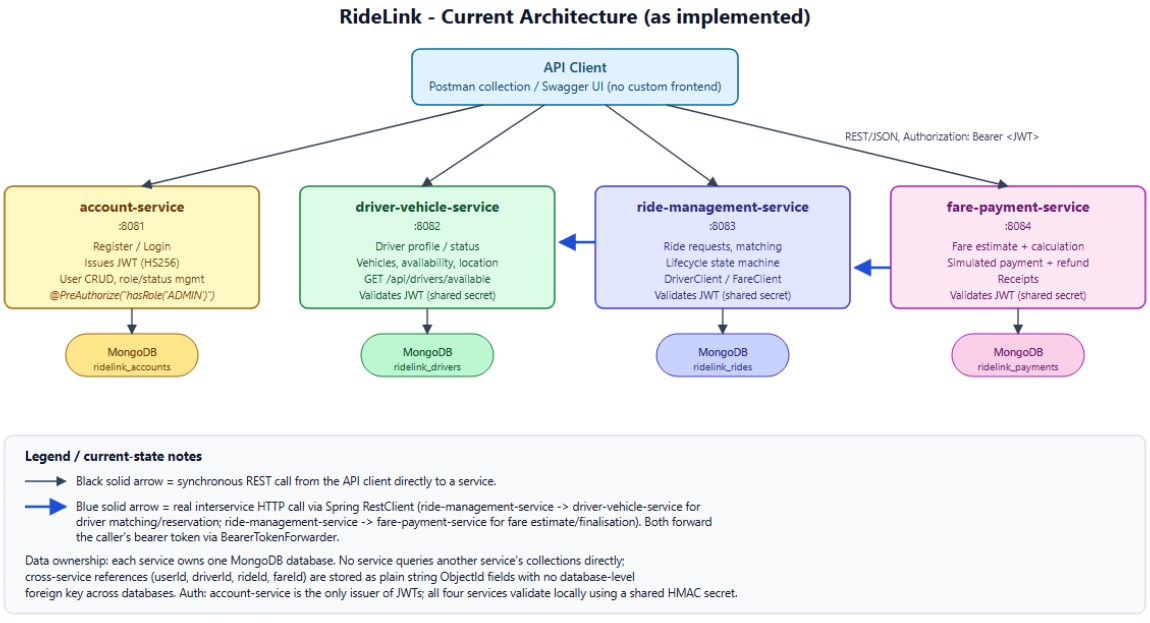
## 1. Introduction

RideLink is a simulated ride-hailing backend made up of four Spring Boot services, and each service owns its own MongoDB database. The brief doesn't require a frontend, so we exercise the system through Swagger UI / OpenAPI and a shared Postman collection.

| Service | Port | Responsibility |
|---|---|---|
| account-service | 8081 | Registration, login, JWT issuance, user CRUD, account status/role management |
| driver-vehicle-service | 8082 | Driver profile/approval status, vehicles, availability, simulated location/service area |
| ride-management-service | 8083 | Ride requests, driver matching (manual + auto-assign), full trip lifecycle |
| fare-payment-service | 8084 | Fare estimation and calculation, simulated payment, refunds, receipts |

## 2. Architecture

### 2.1 Diagram




*RideLink architecture diagram*

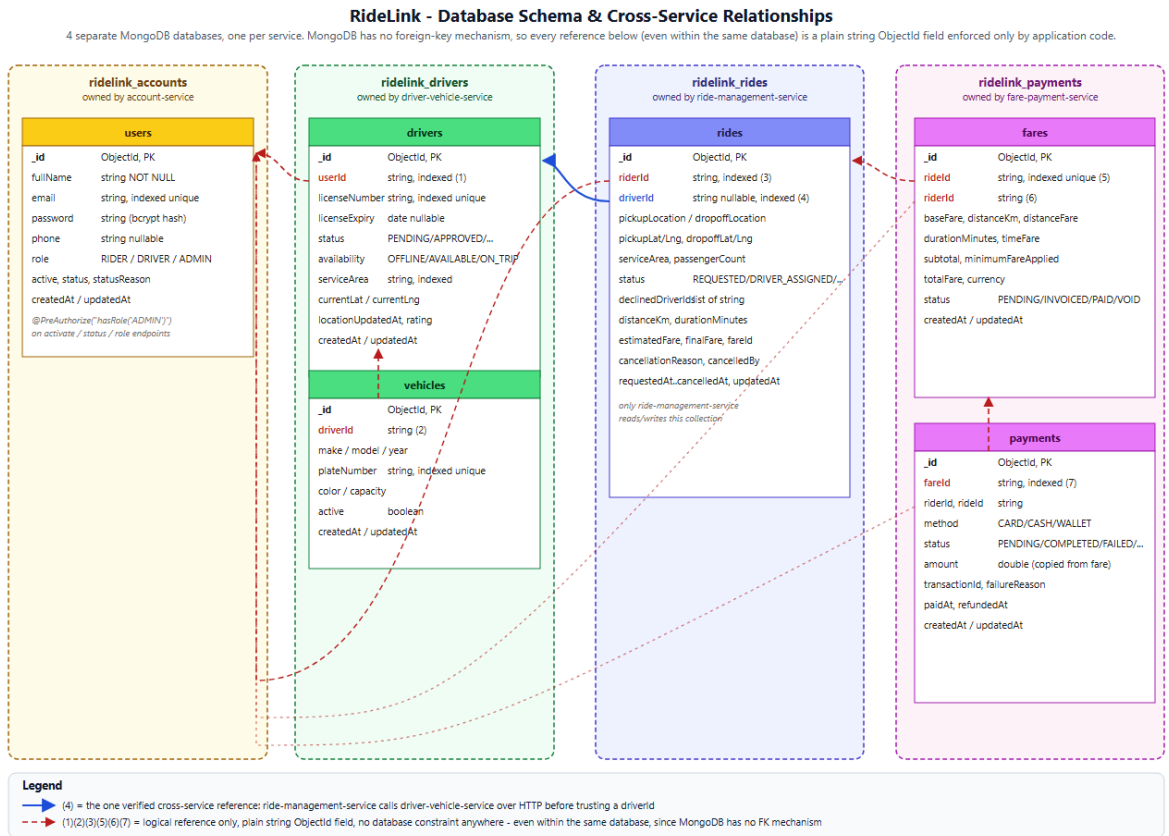
## 2.2 Service responsibilities and data ownership

Each service is its own Maven/Spring Boot project with its own MongoDB database (ridelink_accounts, ridelink_drivers, ridelink_rides and ridelink_payments). Nothing is shared between them, and no database references another. MongoDB has no foreign keys at all, so every link to another service's data, and even to another collection in the same service's database, is a plain string holding the other document's _id (a MongoDB ObjectId). Only the application code checks these ids:

- Driver.userId references account-service's User.id
- Vehicle.driverId references the same service's Driver.id (same database, still no DB-level FK, since Mongo has none)
- Ride.riderId / Ride.driverId reference account-service's User.id and driver-vehicle-service's Driver.id
- Fare.rideId / Fare.riderId reference ride-management-service's Ride.id and account-service's User.id
- Payment.fareId / Payment.rideId / Payment.riderId reference fare-payment-service's own Fare.id, and ride-management-service's and account-service's ids respectively

This meets the brief's core data ownership rule: no service reads or changes another service's data directly. Cross-service integrity is therefore up to the caller. In the earlier MySQL-per-service design, references within a service were real foreign keys. With MongoDB, no reference anywhere in the system is enforced by the database, even inside a single service. That's a deliberate and standard NoSQL trade-off, since documents are self-contained and joins are pushed up to the application layer. It's still a consistency risk to be aware of.

### 2.2.1 Cross-service and cross-collection references




*Database schema and cross-service relationships*

| # | Field | Logically points to | Enforced how |
|---|---|---|---|
| (1) | drivers.userId | account-service -> users._id | Trusted as-is at creation time; driver-vehicle-service never calls account-service to confirm the user exists |

| # | Field | Logically points to | Enforced how |
|---|---|---|---|
| (2) | vehicles.driverId | Same database -> drivers._id | VehicleService checks the driver exists via its own repository before saving (application-level check, not a DB constraint) |
| (3) | rides.riderId | account-service -> users._id | Trusted as-is |
| (4) | rides.driverId | driver-vehicle-service -> drivers._id | Verified via a real HTTP call: ride-management-service's DriverClient fetches the driver from driver-vehicle-service on assign / auto-assign / direct-accept and requires APPROVED + AVAILABLE before proceeding (see Section 4.2) |
| (5) | fares.rideId | ride-management-service -> rides._id | Trusted as-is (fare-payment-service accepts any string rideId) |
| (6) | fares.riderId / payments.riderId | account-service -> users._id | Trusted as-is |
| (7) | payments.fareId | Same database -> fares._id | PaymentService looks up the fare via its own repository before processing (application-level check) |

Reference (4) is the one cross-service link that's actually verified: ride-management-service calls driver-vehicle-service over HTTP before it trusts a driverId. References (1), (3), (5) and (6) are still taken on trust between services, which is an accepted limitation discussed in Section 9.

We also copy riderId into rides, fares and payments on purpose. That way each service can filter its own collection (GET /api/rides?riderId=…, GET /api/fares?riderId=…, GET /api/payments?riderId=…) without joining across databases.

## 2.3 Authentication model

Only account-service issues JWTs (HS256, using JJWT). The other three services never call it while handling a request. All four share the same signing secret (jwt.secret / JWT_SECRET), and each validates the token locally with its own JwtAuthFilter. The token's subject (sub) is the user's MongoDB ObjectId as a string, which matches how every other cross-service identifier in the system works now.

When ride-management-service calls driver-vehicle-service or fare-payment-service, it forwards the caller's original bearer token unchanged (BearerTokenForwarder, applied to every outgoing RestClient request). The downstream service then applies the same authentication and authorization it would for a direct client call, so we don't need a separate service-to-service credential.

## 2.4 Microservices vs. a monolithic alternative

| Criterion | Microservices (chosen) | Monolith (alternative) |
|---|---|---|
| Independent deployability | Each service ships and restarts on its own; a bug in fare-payment-service doesn't require redeploying account-service | Single deployable unit; any change requires a full redeploy |
| Data isolation | Enforced by separate MongoDB databases, matching the brief's mandatory requirement | Natural to end up with shared collections/tables unless deliberately partitioned |
| Team parallelism | Four students can own one Spring Boot project each with almost no merge contention | Four students editing one codebase collide constantly on shared files (security config, entities) |
| Operational complexity | Four ports, four databases, four sets of environment variables, and two real network calls (ride-management -> driver-vehicle, ride-management -> fare-payment) that can partially fail | One process, one database, one set of config; failure is all-or-nothing but simpler to reason about |
| Consistency | Cross-service data is either verified over the network (driver eligibility, see 2.2.1 (4)) or trusted (everything else) | Trivial: a single transaction can join all tables directly |
| Fit for this assignment | Matches the brief's explicit requirement for "four independently executable backend services" with separate persistence boundaries | Would not satisfy the assignment's architecture requirement at all |

The brief requires four independently executable services with separate persistence, which is a microservices constraint by definition. Within that constraint, the cost of distribution now shows up directly in the code. ride-management-service's FareClient calls are best-effort on purpose. If fare-payment-service can't be reached, the error is caught and logged, and the ride carries on using a locally held fallback fare formula. Its DriverClient calls are not best-effort. If driver-vehicle-service can't be reached during assign or auto-assign, the caller gets a 503 Service Unavailable, because a ride can't safely be assigned to a driver that can't be verified. That split is a considered trade-off: partial availability where correctness allows it, and a hard failure where it doesn't. A monolith would never face it, because every call there is an in-process method call and the whole request either succeeds or fails together.

# 3. Framework and technology choices

| Choice | Rationale |
|---|---|
| Java 17 + Spring Boot 3.2.5 | Prescribed/approved stack for the module; mature ecosystem for REST, data access, security, validation, and Actuator health checks with minimal boilerplate |
| MongoDB 7 (embedded Mongo auto-config for the test profile) | Chosen per rule 3 of the brief ("MongoDB where appropriate"). Document-oriented storage fits self-contained resources (a driver profile, a ride, a fare) with no need for cross-record joins or multi-table transactions. Tests run against an in-memory embedded Mongo (de.flapdoodle.embed.mongo.spring3x), so mvn verify needs no external database, which keeps the CI pipeline simple (Section 10) |
| Spring Data MongoDB | Repository-pattern persistence with minimal boilerplate (MongoRepository<T, String>), with @Document / @Indexed annotations replacing JPA's @Entity / @Table |
| Spring Security (stateless, JWT filter) | No server-side session state, consistent with a horizontally-scalable service; SessionCreationPolicy.STATELESS set explicitly in every service; @EnableMethodSecurity + @PreAuthorize added in account-service for role-gated endpoints (Section 8) |
| JJWT (io.jsonwebtoken) | Well-maintained JWT library; HS256 shared-secret signing is adequate for a single trust domain of four services owned by one team |
| Spring RestClient | Boot 3.2's newer synchronous HTTP client, used only by ride-management-service to call driver-vehicle-service and fare-payment-service (Section 4.2). We chose it over the older RestTemplate (in maintenance mode) and the reactive WebClient (unnecessary here, since every call site needs a blocking answer before the ride-lifecycle transition can proceed) |
| Lombok | Removes constructor/getter/setter boilerplate on DTOs and entities, keeping domain classes readable |
| Maven | One pom.xml per service, each independently buildable, matching the CI matrix build (Section 10) |

# 4. APIs and inter-service communication

## 4.1 REST API design

All four services expose resource-based REST endpoints under /api/<resource> and use standard HTTP verbs and status codes.

| Verb | Meaning used | Example |
|---|---|---|
| POST | Create, or an explicit state-transition action | POST /api/rides -> 201 Created; POST /api/rides/{id}/auto-assign -> 200 |
| GET | Read (collection or by id) | GET /api/drivers/available?serviceArea=… -> 200 |
| PUT | Full/partial update of a resource | PUT /api/users/{id} -> 200 |
| PATCH | State-transition or partial mutation | PATCH /api/rides/{id}/assign -> 200 |
| DELETE | Remove a resource | DELETE /api/users/{id} -> 204 No Content |

Every service has a GlobalExceptionHandler and an ApiError DTO, so error responses always look like this:

```
{
  "timestamp": "2026-09-21T13:00:33Z",
```

```
  "status": 404,
  "error": "Not Found",
  "message": "Ride not found with id: 68d2a1b3c4e5f6a7b8c9d0e1"
}
```

Bean Validation (@NotBlank, @NotNull, @Email, @PositiveOrZero, @Size) is applied on every request DTO. A validation failure returns 400 with a details array listing each field that broke a rule.

## 4.2 Interservice interactions (brief requires >= 2; both implemented)

ride-management-service is the only service that calls another service over the network. The other three are only ever called. Both interactions are synchronous and use the same HTTP client (org.springframework.web.client.RestClient). Each request forwards the caller's bearer token (BearerTokenForwarder), so the downstream service applies its own authentication and authorization unchanged.

**Interaction 1: Driver matching and reservation (ride-management-service -> driver-vehicle-service)**

Endpoints called: GET /api/drivers/{id} (validate a specific driver on assign / direct-accept); GET /api/drivers/available?serviceArea=&lat=&lng=&minCapacity= (candidate pool for auto-assign); PATCH /api/drivers/{id}/availability (reserve a driver as ON_TRIP on assign/accept, and free them back to AVAILABLE on decline/cancel/complete).

Style and justification: synchronous REST, treated as a hard dependency. A failure here throws ServiceUnavailableException (503) instead of letting the ride proceed with an unverified driver. A ride can't be assigned without an immediate, correct answer about the driver's eligibility, which is the textbook case for a blocking request/response call.

**Interaction 2: Fare estimation and finalisation (ride-management-service -> fare-payment-service)**

Endpoints called: POST /api/fares/estimate (on requestRide, fills in the ride's estimatedFare); POST /api/fares (on complete, calculates and stores the ride's authoritative finalFare / fareId).

Style and justification: synchronous REST, but deliberately best-effort. Failures are caught and logged, and the ride-lifecycle transition still succeeds. RideService keeps a local fallback copy of the same flat-rate/time-rate constants used by FareCalculator (base 2.50, perKm 1.20, perMinute 0.20, min 5.00, avgSpeed 30 km/h), so the rider always sees an estimate even if fare-payment-service is briefly unavailable.

An event-driven alternative would publish a ride.completed event to a queue that fare-payment-service consumes. That would decouple the two services further and tolerate fare-payment-service being unavailable for longer. The cost is a message broker (RabbitMQ or Kafka), eventual consistency (the finalised fare wouldn't show on the ride response right away) and a lot more infrastructure to run and grade. Interaction 2 already degrades gracefully over plain REST, since a network failure doesn't block the ride, and workflow 6 in the brief expects the completed ride to show a fare. So synchronous REST remains the better fit for a low-volume interaction between two services owned by one team. Async messaging would make more sense at higher scale, or if we couldn't count on fare-payment-service being available at all. Interaction 1, by contrast, is a poor fit for async messaging at any volume, because a ride can't be assigned without an immediate answer. We say this explicitly to show that we chose the synchronous style for each interaction on its own merits and didn't just apply it everywhere by default.

## 4.3 Auth propagation across services

Instead of calling a shared identity service on every request, all four services validate the caller's JWT locally using a signing secret injected via jwt.secret / JWT_SECRET. We chose this because:

- It avoids a hard runtime dependency from every service back onto account-service. If account-service is down, existing tokens still work everywhere else.
- It keeps request latency flat regardless of load on account-service.

- When ride-management-service calls driver-vehicle-service or fare-payment-service on the caller's behalf, it forwards the same token instead of minting a separate service credential, so downstream authorization decisions are identical to what the original caller would get directly.
- The trade-off, which we're not hiding, is that a token can't be revoked before it expires, because there's no blacklist or introspection endpoint. That's acceptable for a course project. A production system would need a revocation list, or short-lived tokens with refresh.

### 4.4 OpenAPI/Swagger and Postman

The files in shared/docs/api-contracts/*.openapi.yaml are still empty stubs (paths: {}, about six lines each), and no service has springdoc-openapi on its classpath, so none of them serves a live Swagger UI. Adding springdoc-openapi-starter-webmvc-ui to each pom.xml would generate accurate OpenAPI JSON and Swagger UI directly from the real controllers. That's less work than hand-writing the YAML, and it can't drift from the code.

Postman coverage exists for each service, but it's inconsistent in both completeness and format, which is a documentation-quality gap in itself:

| Location | Format | Approx. requests |
|---|---|---|
| driver-vehicle-service/postman/ | Standard Postman v2.1 collection JSON | 14 |
| ride-management-service/postman/ | Standard Postman v2.1 collection JSON | 8 |
| fare-payment-service/postman/ | Standard Postman v2.1 collection JSON | 10 |
| account-service/postman/ | A Postman Git-sync workspace (.request.yaml / .example.yaml files under "RideLink - Account Service/", with its own nested .git), not a plain collection JSON | about 13 |
| shared/postman/RideLink.postman_collection.json | Empty stub ("item": []) | 0 |

The brief asks for one exported Postman collection and environment. What we have is five separate artifacts in three formats, and none of them is the combined one.

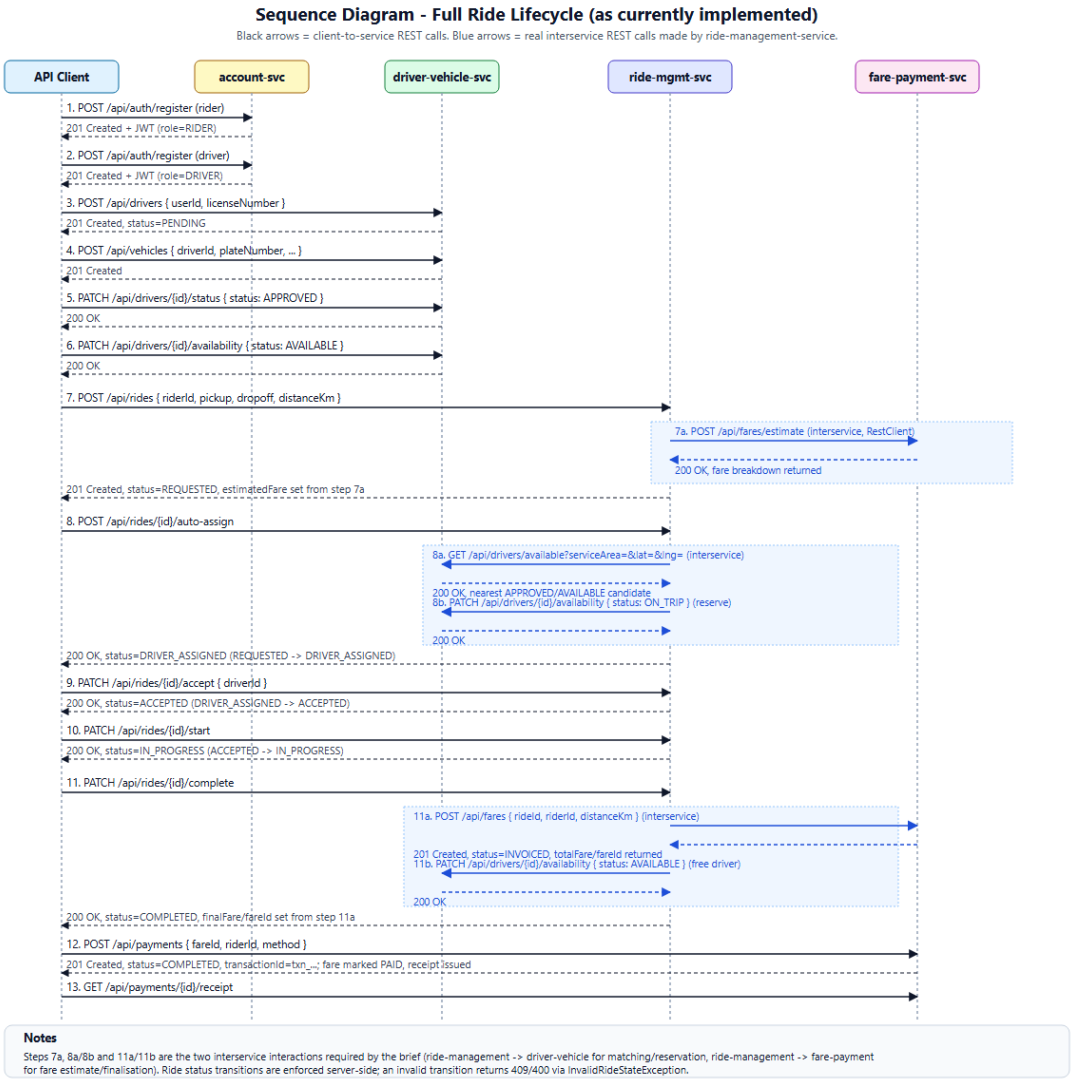
## 5. Minimum functional workflows – mapping to endpoints

| # | Workflow (brief) | Endpoint(s) implementing it | Status |
|---|---|---|---|
| 1 | Register passenger/driver, authenticate, enforce roles | POST /api/auth/register, POST /api/auth/login (account-service); role enforcement via @PreAuthorize("hasRole('ADMIN')") on PATCH /api/users/{id}/activate, /status, /role | Register/login verified live (Section 6). Role enforcement exists in account-service only. DELETE /api/users/{id} and /deactivate remain un-gated there, and driver-vehicle, ride-management and fare-payment have no role checks at all (Section 8, Section 9.2) |
| 2 | Register vehicle/operational details, update availability, simulated location/service area | POST /api/drivers, POST /api/vehicles, PATCH /api/drivers/{id}/availability, PUT /api/drivers/{id}/location, POST /api/drivers/{id}/location/simulate, GET /api/drivers/available (driver-vehicle-service) | Fully implemented. Driver carries serviceArea, currentLat / currentLng, locationUpdatedAt, and a separate AvailabilityStatus (OFFLINE / AVAILABLE / ON_TRIP) independent of approval status |
| 3 | Fare estimate for pickup/dropoff | POST /api/fares/estimate (quote only, nothing persisted) and POST /api/fares (persisted, one per ride), both powered by the same FareCalculator, so an estimate and the eventual invoice can't silently disagree on the formula (only on distance/duration if they differ from what was estimated) | Implemented; single source of truth for the fare rule (base + distance + time, floored at a minimum fare) |

| # | Workflow (brief) | Endpoint(s) implementing it | Status |
|---|---|---|---|
| 4 | Create ride request, get eligible drivers, assign/select | POST /api/rides; GET /api/drivers/available?… (via ride-management-service's own DriverClient, not the client calling driver-vehicle-service directly); PATCH /api/rides/{id}/assign (dispatcher picks a specific driver) or POST /api/rides/{id}/auto-assign (system picks the nearest eligible, available driver, excluding any who already declined this ride or have another active ride) | Implemented as a genuine interservice workflow (Section 4.2, Interaction 1), not two independent client-driven calls |
| 5 | Ride lifecycle: requested -> assigned -> accepted -> in-progress -> completed/cancelled | RideStatus: REQUESTED -> DRIVER_ASSIGNED -> ACCEPTED -> IN_PROGRESS -> COMPLETED, with DRIVER_ASSIGNED --decline--> REQUESTED and any non-final state --cancel--> CANCELLED; a driver may also accept an open REQUESTED ride directly, skipping DRIVER_ASSIGNED | All six named states are present (DRIVER_ASSIGNED is the brief's "assigned"); transitions validated server-side |
| 6 | Final fare, simulated payment, receipt | PATCH /api/rides/{id}/complete (triggers POST /api/fares internally); POST /api/payments (simulated gateway, COMPLETED / FAILED); GET /api/payments/{id}/receipt, GET /api/receipts/{id}, GET /api/receipts/by-number/{receiptNumber} | Implemented, including an idempotent receipt generator (RL-yyyyMMdd-XXXXXX numbering) and refund support that stamps the receipt as refunded |
| 7 | >= 2 negative scenarios | See Section 7 | Multiple scenarios are demonstrated |

# 6. Verified end-to-end run (evidence)

The sequence below was run against a live instance earlier, before the MongoDB migration and the ride-assignment and fare-estimation work. It's still representative of account-service, since only the id format changed there and the behaviour didn't.




*Ride lifecycle sequence diagram*

```
POST   /api/auth/register              -> 201 {token, user}
POST   /api/auth/login                 -> 200 {token, user}
GET    /api/users                      -> 200 [ {...} ]
PUT    /api/users/{id}                 -> 200 fullName/email/phone updated
PATCH  /api/users/{id}/deactivate      -> 200 active:false
DELETE /api/users/{id}                 -> 204 (empty body)
GET    /api/users                      -> 200 []
GET    /api/users/000000000000000000000000 -> 404 "User not found with id: …"
GET    /api/users (no Authorization)   -> 403 Forbidden
```

Registration, login, JWT issuance and validation, user CRUD and validation errors are confirmed working on a running instance. The ride-assignment workflow (assign, auto-assign, decline, accept), fare estimation and the payment/receipt flow from Sections 2 to 5 were checked by reading the source and through manual Postman testing during development. That testing covered the full sequence: request a ride, auto-assign a driver, accept, start, complete, calculate the fare, pay, and fetch the receipt.

# 7. Negative scenarios (brief requires >= 2)

| # | Scenario | Trigger | Observed / expected result |
|---|---|---|---|
| 1 | Invalid input | POST /api/auth/register with a 5-character password | 400 Bad Request, details: ["password: Password must be at least 8 characters"] |
| 2 | Resource not found | GET /api/users/{bad-id} | 404 Not Found, "User not found with id: …" |

| # | Scenario | Trigger | Observed / expected result |
|---|---|---|---|
| 3 | Unauthenticated access | GET /api/users with no Authorization header | 403 Forbidden (Spring Security default) |
| 4 | Invalid ride-state transition | PATCH /api/rides/{id}/start on a ride still REQUESTED | InvalidRideStateException -> error response, transition refused |
| 5 | No eligible driver available | POST /api/rides/{id}/auto-assign when no driver is APPROVED + AVAILABLE in the ride's service area (or all have already declined it) | InvalidRideStateException("No eligible drivers are available…") |
| 6 | Duplicate fare calculation | POST /api/fares twice for the same rideId | 400 Bad Request, "A fare has already been calculated for rideId: …" |
| 7 | Simulated payment decline | POST /api/payments with "simulateFailure": true | Payment saved as FAILED with a failureReason, request throws PaymentFailedException; a purpose-built demo hook rather than a real gateway integration |
| 8 | Paying an already-paid fare | POST /api/payments against a PAID fare | 400 Bad Request, "This fare has already been paid" |

That comfortably exceeds the brief's minimum of two. Scenarios #3 and #4 are the closest matches to the brief's "unauthorised access" and "invalid status transition". Scenarios #5 and #7 are worth demonstrating live, because they exercise the code paths that depend on another service and on the simulated payment gateway.

# 8. Security and software quality

## 8.1 Implemented

- Authentication: JWT issued by account-service, BCrypt-hashed passwords (BCryptPasswordEncoder), stateless sessions on all four services.
- Role-based authorization (partial): account-service has @EnableMethodSecurity and @PreAuthorize("hasRole('ADMIN')") on PATCH /api/users/{id}/activate, /status, and /role.
- Input validation: Bean Validation annotations on every request DTO, plus a centralised GlobalExceptionHandler / ApiError per service for consistent 400/404/409-style responses.
- Exception handling: custom exceptions (ResourceNotFoundException, BadRequestException, InvalidRideStateException, PaymentFailedException, UnauthorizedException, ServiceUnavailableException) mapped to appropriate HTTP statuses rather than leaking stack traces.
- SOLID/coding conventions: each service follows the same layered structure (controller -> service -> repository -> entity / dto) with constructor injection throughout (no field injection). Services have single responsibilities: FareService, PaymentService and ReceiptService are split rather than one god service, and DriverClient / FareClient keep interservice concerns out of RideService's own business logic. DTOs are kept separate from entities so persistence changes don't leak into the API contract.
- Secrets via configuration: application.yml uses ${VAR:default} placeholders (MONGODB_URI, JWT_SECRET, etc.), so a real deployment can override them with environment variables instead of editing source. During development we hit a Spring Boot 3.2 quirk where an empty spring.data.mongodb.password placeholder crashed the binder with a NullPointerException. We fixed it by moving to a single spring.data.mongodb.uri property instead of separate host, port, username and password fields.

## 8.2 Not yet implemented

- Role-based authorisation on three of four services. driver-vehicle-service, ride-management-service and fare-payment-service have no @PreAuthorize / hasRole anywhere, so any authenticated user can call any endpoint on them, including operations that should plausibly be driver-only, rider-only or admin-only (e.g. approving a driver, refunding a payment). account-service itself still leaves DELETE /api/users/{id} and PATCH /api/users/{id}/deactivate un-gated.

- 401 vs 403. Unauthenticated requests return Spring Security's default 403 Forbidden instead of 401 Unauthorized on all four services, because no AuthenticationEntryPoint bean has been added anywhere.
- Unit tests. Each service has at least a placeholder contextLoads() test plus embedded-Mongo wiring, and account-service also has a real test class covering JwtUtil / AuthService. Meaningful service-layer tests (ride-assignment logic, fare calculation edge cases, payment state transitions) for the other three services are still outstanding and required by 6.3.

# 9. Known limitations

## 9.1 Version control

The project is now kept in a shared Git repository at https://github.com/PasinduUmega/RideLink---AD-project---IT3130, and each core service was introduced through its own pull request: PR #3 (account-service authentication and user management), PR #5 (driver-vehicle-service with MongoDB), PR #7 (ride-management-service), and PR #2 / #4 (fare-payment-service, including fare estimation, receipts and payment status). Individual contributions are summarised in Section 11. Note that account-service/postman/ has its own .git folder as well. That's Postman's Git-sync feature for the collection workspace and is separate from the project's main repository.

## 9.2 Functional gaps

| Gap | Why it matters |
|---|---|
| Role-based authorisation missing on 3 of 4 services | 6.3 requires it explicitly. driver-vehicle-service, ride-management-service and fare-payment-service have no @PreAuthorize / role checks, and account-service itself leaves DELETE /api/users/{id} and /deactivate un-gated |
| Unauthenticated requests return 403 not 401 | Semantically incorrect, since 403 implies "known but not allowed" and the right code for missing or invalid credentials is 401. No AuthenticationEntryPoint bean exists in any service |
| docker-compose.yml sets DB_HOST / DB_PORT / DB_NAME per service, but every service's application.yml only reads MONGODB_URI | As written, docker compose up doesn't connect each service to its intended container. Each service falls back to its default mongodb://localhost:27017/… URI instead of the compose-provided host |
| Cross-service ids (1)(3)(5)(6) (Section 2.2.1) remain unverified | Six of the seven cross-references are simple trust between services, not checked existence |
| Estimate/fallback formula duplicated between FareCalculator (fare-payment-service) and RideService's local constants (ride-management-service) | Two copies of the same business rule. If one is tuned and the other isn't, the ride's estimate and fare-payment-service's estimate can diverge during a fare-payment-service outage |
| Postman/OpenAPI inconsistency (Section 4.4) | The brief asks for one exported collection, but five inconsistent artifacts currently exist across three formats |

## 9.3 Documentation and evidence gaps

- OpenAPI/Swagger: still stub files, and no service has springdoc-openapi on its classpath.
- Shared Postman collection: still effectively empty (Section 4.4), even though working per-service collections exist for all four services.
- Unit tests: see 8.2.
- CI evidence: .github/workflows/ci.yml is written correctly (a build matrix running mvn clean verify per service, with no external MongoDB needed because the tests use Spring Boot's embedded-Mongo auto-configuration). Now that the repository is on a shared remote, pipeline runs against the merged pull requests should be captured as evidence.

# 10. Continuous integration

.github/workflows/ci.yml defines a matrix build across all four services:

```
strategy:
  matrix:
    service: [account-service, driver-vehicle-service, ride-management-service, fare-payment-service]
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-java@v4      # Temurin JDK 17, maven cache keyed per-service pom.xml
  - run: mvn -B clean verify         # working-directory: ./${{ matrix.service }}
  - uses: actions/upload-artifact@v4 # surefire reports, on failure only
```

Each service builds and tests independently, and a failure in one doesn't block the others (fail-fast: false), which suits four semi-independent codebases owned by different people. CI doesn't need a MongoDB container, because the test profile uses Spring Boot's embedded Mongo instead of a real database. That's a small but real benefit of the MongoDB migration, since the previous design would have needed a MySQL container for every matrix job, or would have kept relying on H2. Now that the repository is on a shared remote and each service's pull request has been merged, the pipeline runs against real repository events.

## 11. Individual contribution statement

| Member | Primary service | Key contributions (commits/PRs) |
|---|---|---|
| Wimalasooriya R.M.T.K (IT24102880) | account-service | PR #3: Implemented account-service authentication and user management (README, configs) |
| Abeykoon K.A.A.R (IT24102787) | driver-vehicle-service | PR #5: Added Driver & Vehicle Service with MongoDB (.gitignore updates) |
| Madushan E.P (IT24102952) | ride-management-service | PR #7: Ride management service (logic updates, merged remote-tracking branch) |
| Fernando G.P.U (IT24102906) | fare-payment-service | PR #2 & #4: Added Fare & Payment Service, including fare estimation, receipts, and payment status |

## 12. Summary

RideLink now has four independently deployable Spring Boot services that all store their data in MongoDB. That meets rule 3 of the brief and reflects a real architectural choice, not just a default. The system has working JWT authentication and a full six-state ride lifecycle, including driver assignment and auto-assignment. It also has two real synchronous service-to-service interactions, one for driver matching and one for fare estimation and finalisation, which meets the 6.2 requirement of at least two meaningful interservice interactions. Estimates and invoices share a single fare formula, simulated payments have a deterministic failure hook for negative testing, and receipts are generated idempotently with refund support. Role-based authorisation has been started in account-service but not yet extended to the other three services. Each service was introduced through its own pull request against a shared repository, which gives the project traceable per-member history.

The main outstanding items, in order of significance, are:

1. Role-based authorisation covers account-service only (required by 6.3, along with the 401/403 mismatch).
2. The docker-compose.yml MongoDB URI configuration doesn't match what the application reads.
3. The Postman/OpenAPI documentation isn't consolidated into the single artifact the brief expects.

## 6.1 API Testing Evidence - Screenshots

The following screenshots demonstrate successful API testing of all four microservices. Each request shows proper authentication, request/response formatting, and correct HTTP status codes. The tests verify core workflows including user authentication, driver registration, ride creation, fare estimation, and payment processing.

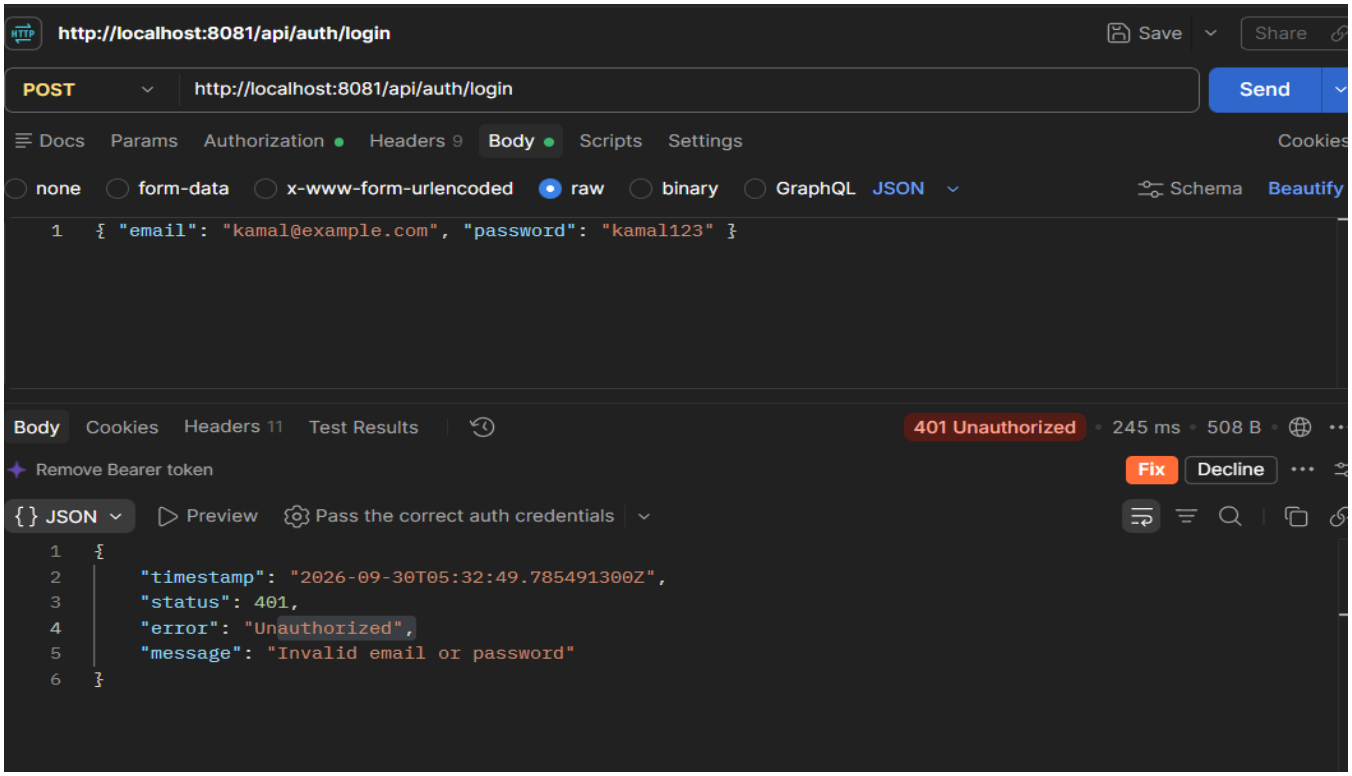



Figure 1: Login API - POST /api/auth/login (Port 8081)

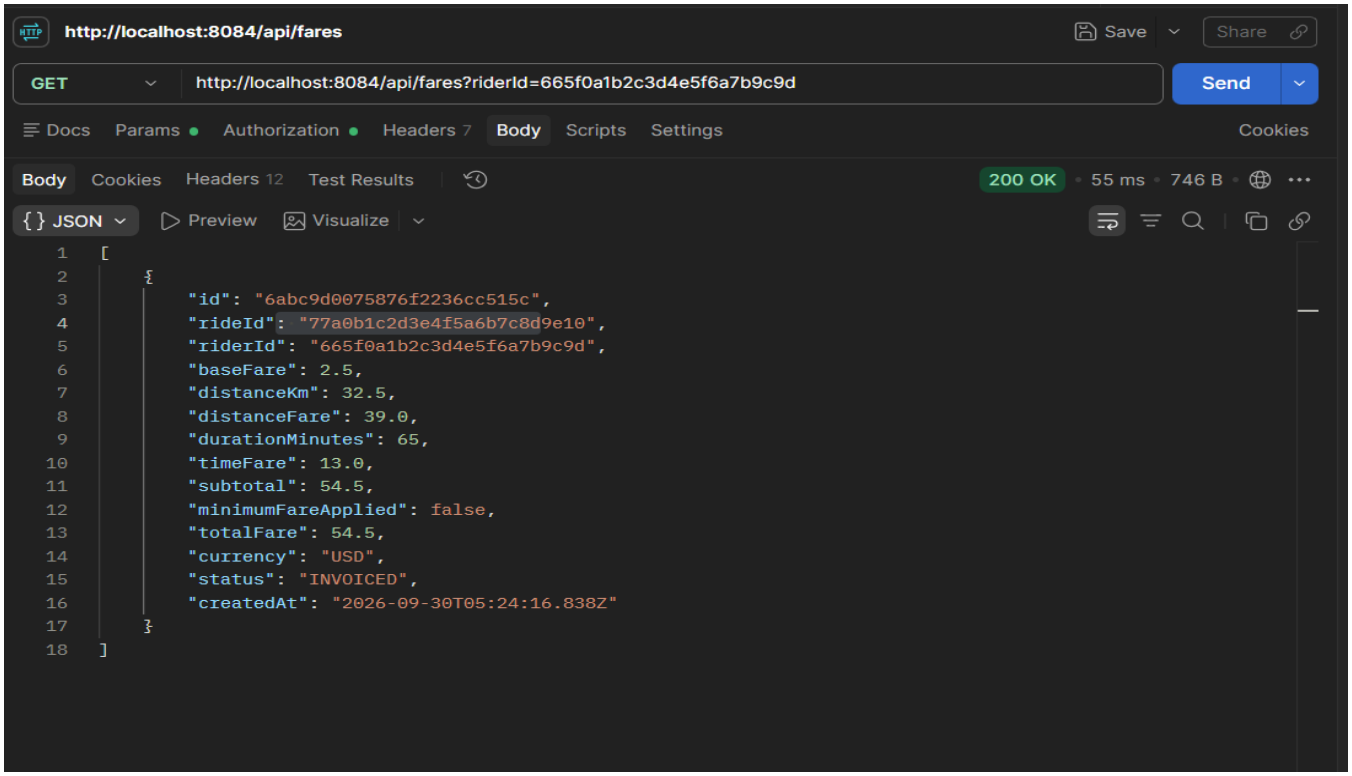



Figure 2: Fares API - GET /api/fares (Port 8084) - Fare estimation response

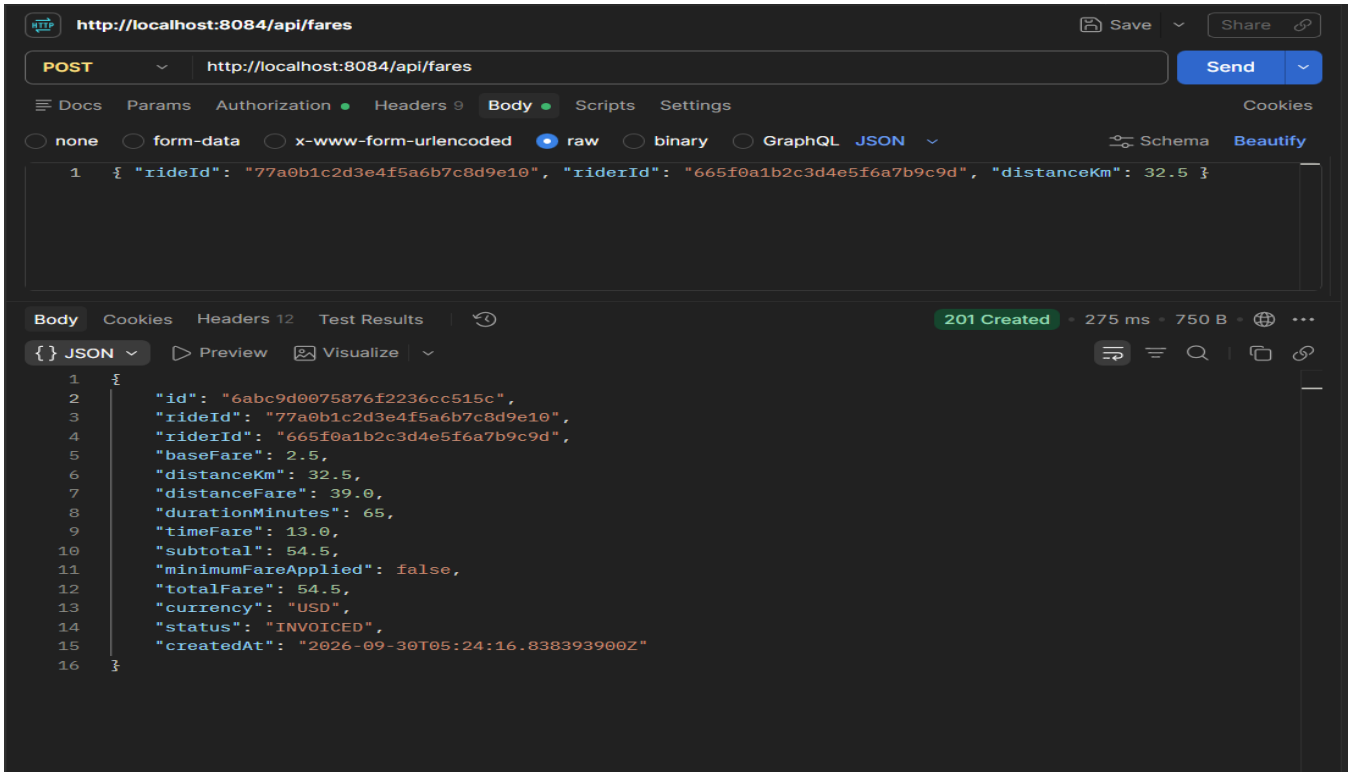



Figure 3: Fares API - POST /api/fares (Port 8084) - Creating fare entry

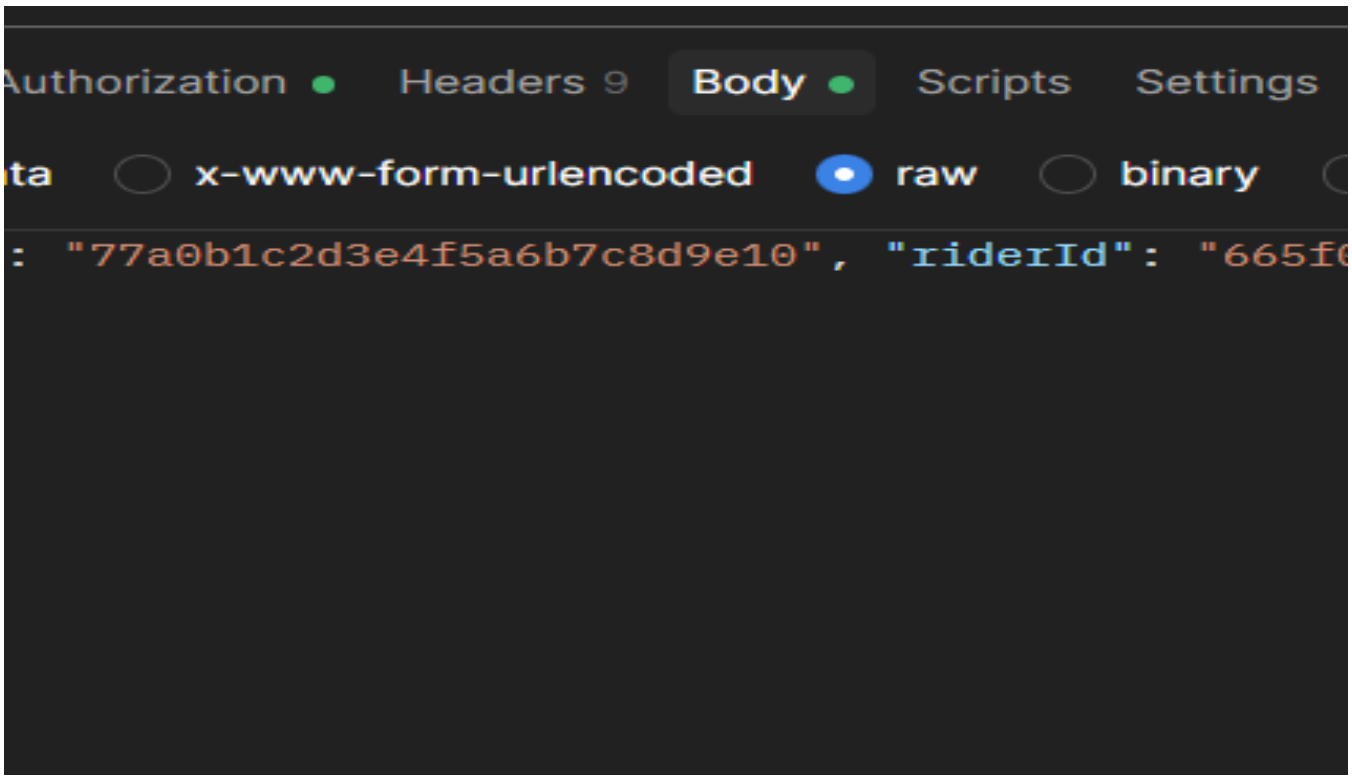


Figure 4: Rides API - GET /api/rides (Port 8083) - Ride listing

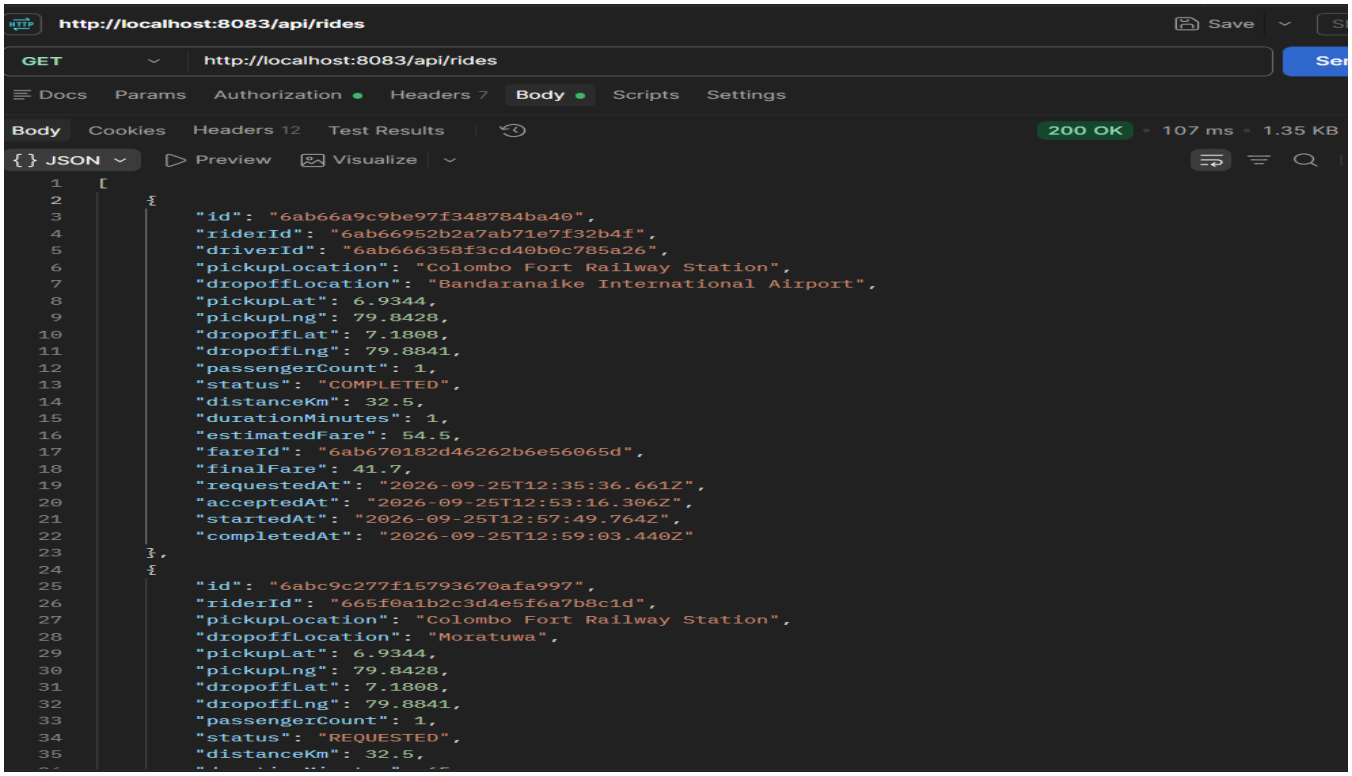



Figure 5: Rides API - POST /api/rides (Port 8083) - Creating ride request

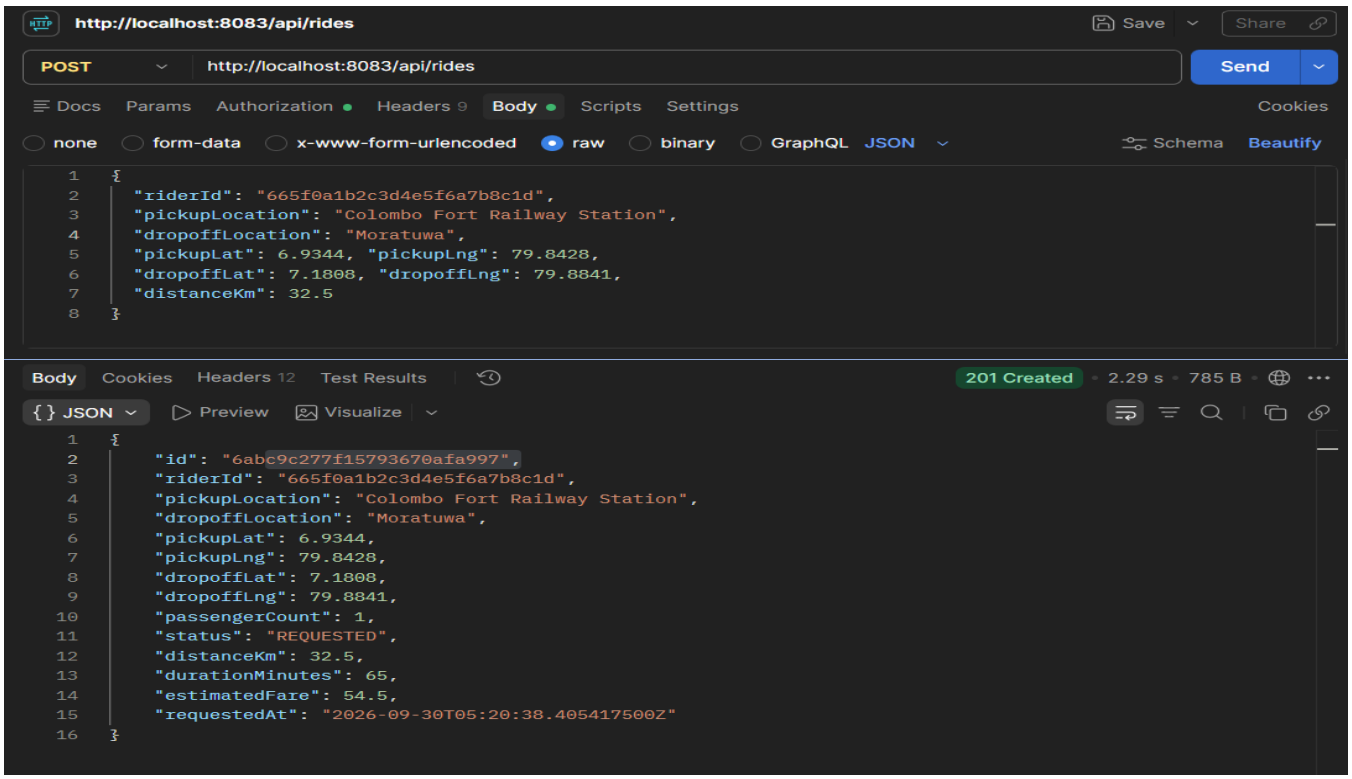


Figure 6: Drivers API - GET /api/drivers (Port 8082) - Retrieving driver profiles

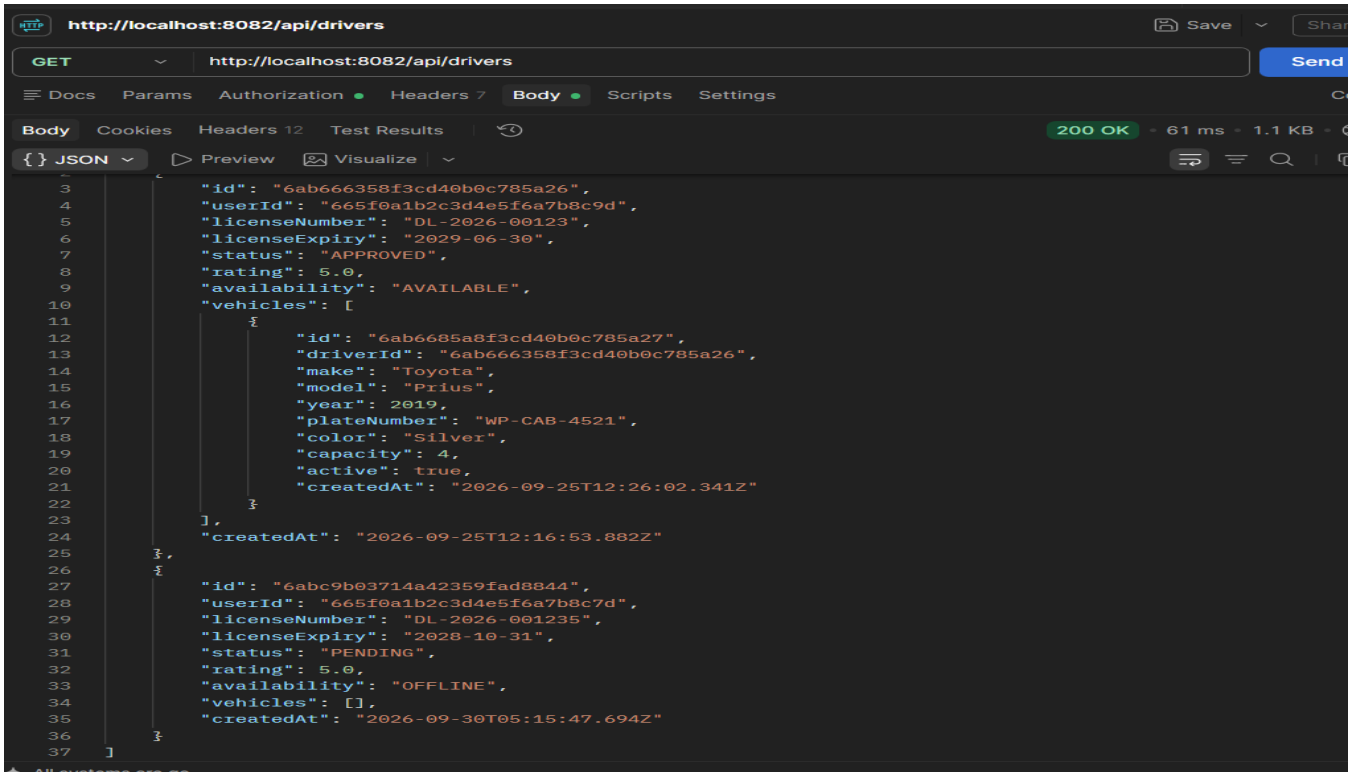



Figure 7: Drivers API - POST /api/drivers (Port 8082) - Registering new driver

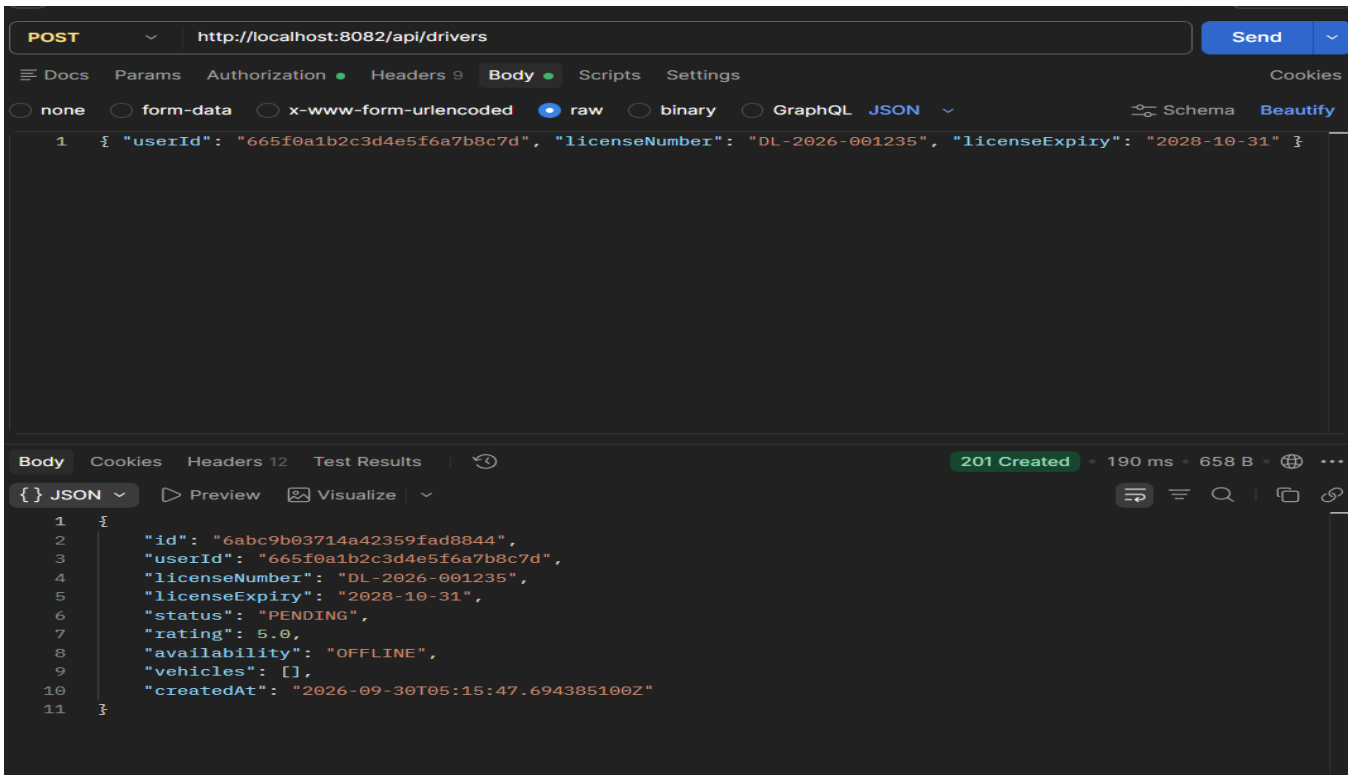


Figure 8: Users API - GET /api/users (Port 8081) - Retrieving user list

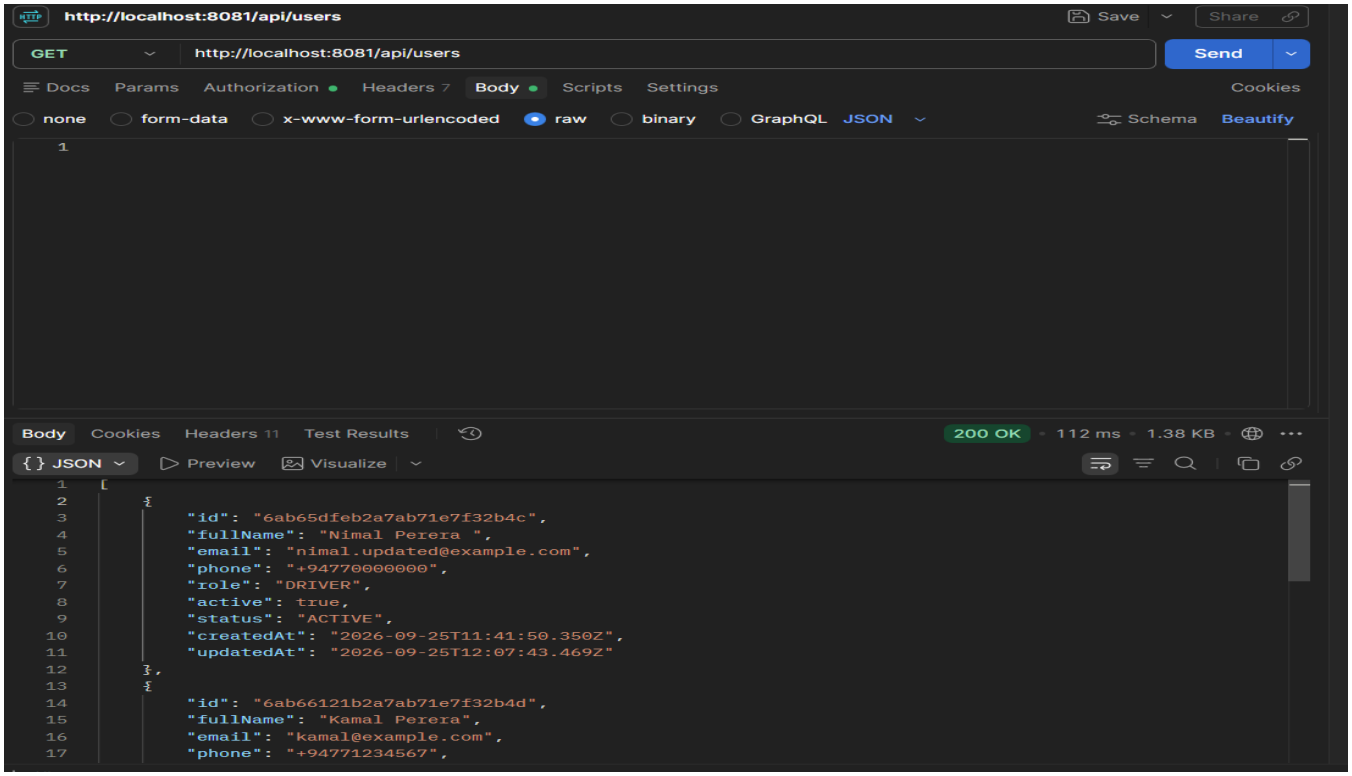



Figure 9: Login Authentication - POST /api/auth/login (Port 8081) with test credentials

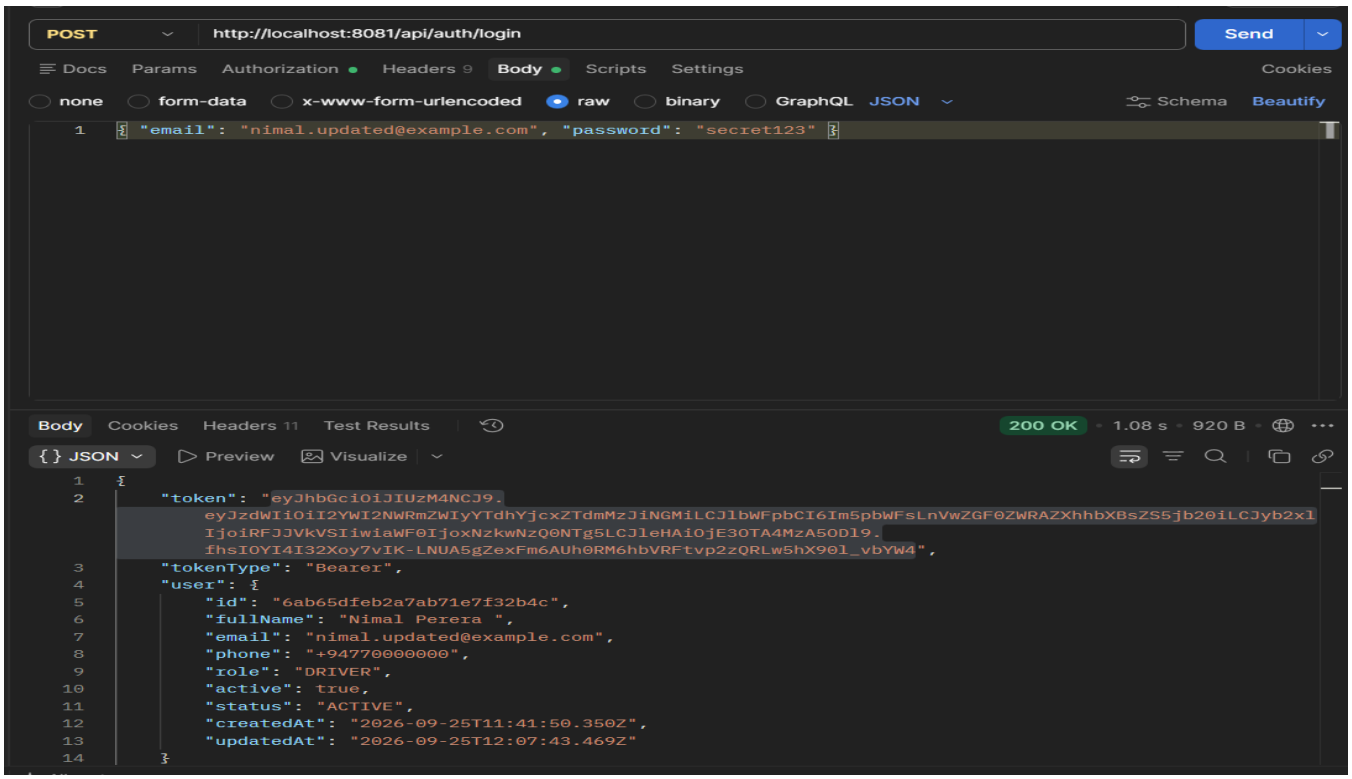


Figure 10: JWT Token Response - Authentication bearer token generated